Blockchain

Dr. Muhammad Umar Janjua **(Cambridge, UK)**

Office: 4th Floor (Blockchain Lab)

Email: umar.janjua@itu.edu.pk

Teaching Assistants: Fatima Shaheen, Abeeha Fatima

Emails: mscs23019@itu.edu.pk, mscs22009@itu.edu.pk

Zero knowledge proof

Agenda

Quick Examples for the Concept of **"Zero Knowledge"**

zkSNARK Construction

Use Cases Utilizing **ZKP**

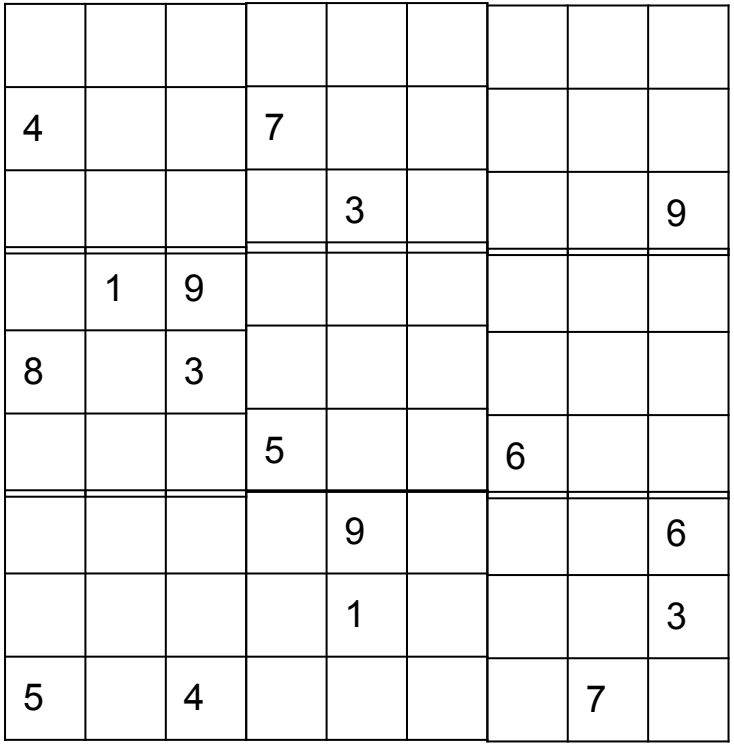
Further Readings and Hands-on Tutorials

Scenario: Sudoku puzzle has a solution

Alice: hey Bob, here's a new Sudoku puzzle,

I designed yesterday, would you like to try it?

Bob: I'm interested only if it really has a solution!



"Zero Knowledge"

Alice wants to convince Bob of something

- The Sudoku puzzle has a solution

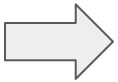
Bob should not learn "too much"

- The Sudoku solution

How To Convince Bob the Puzzle Has a Solution?

Step 1: Alice to hide the solution by using different numbers, 1->9, 7->8, 9->2, etc.

9	3	2	1	4	8	5	6	7
4	6	8	7	5	9	3	1	2
1	7	5	6	3	2	4	8	9
6	1	9	2	7	4	8	3	5
8	5	3	9	6	1	7	2	4
2	4	7	5	8	3	6	9	1
3	8	1	4	9	7	2	5	6
7	2	6	8	1	5	9	4	3
5	9	4	3	2	6	1	7	8

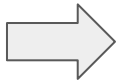


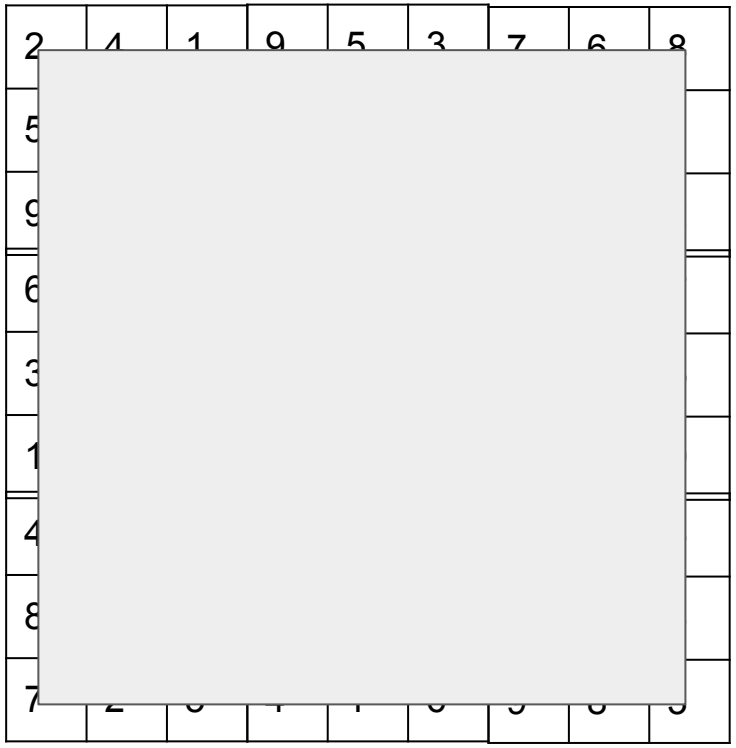
2	4	1	9	5	3	7	6	8
5	6	3	8	7	2	4	9	1
9	8	7	6	4	1	5	3	2
6	9	2	1	8	5	3	4	7
3	7	4	2	6	9	8	1	5
1	5	8	7	3	4	6	2	9
4	3	9	5	2	8	1	7	6
8	1	6	3	9	7	2	5	4
7	2	5	4	1	6	9	8	3

How To Convince Bob the Puzzle Has a Solution?

Step 2: Alice then masks the solution, so it's ready to be presented to Bob

9	3	2	1	4	8	5	6	7
4	6	8	7	5	9	3	1	2
1	7	5	6	3	2	4	8	9
6	1	9	2	7	4	8	3	5
8	5	3	9	6	1	7	2	4
2	4	7	5	8	3	6	9	1
3	8	1	4	9	7	2	5	6
7	2	6	8	1	5	9	4	3
5	9	4	3	2	6	1	7	8

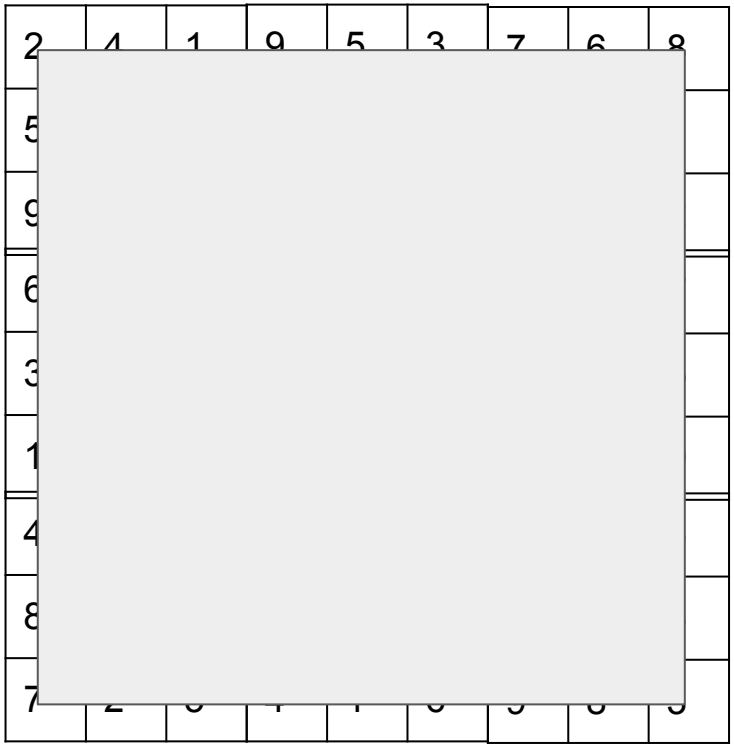




How To Convince Bob the Puzzle Has a Solution?

Step 3: the masked solution is presented to Bob

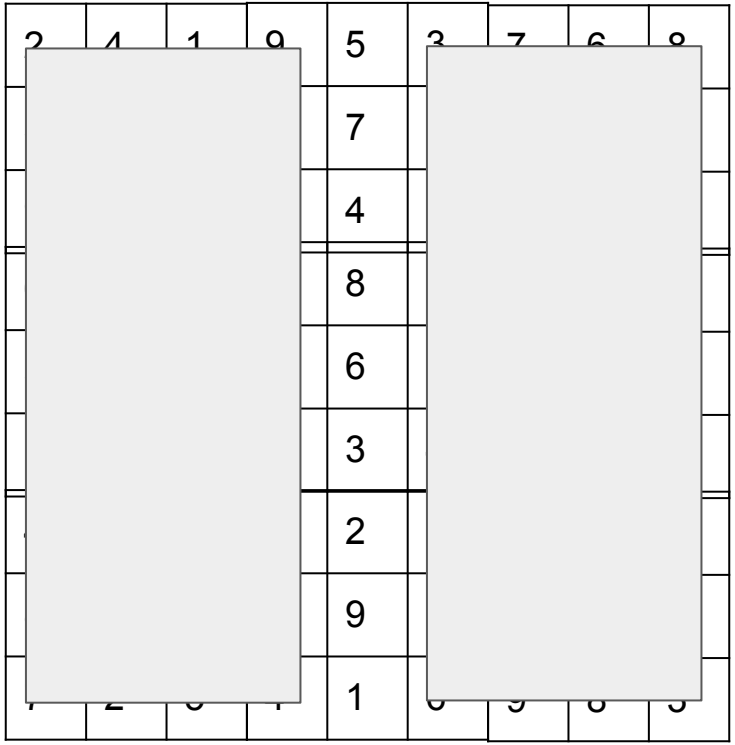
Alice



Bob

How To Convince Bob the Puzzle Has a Solution?

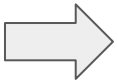
Step 4: Bob randomly picks a unit (row, column or 3x3 square) and asks Alice to reveal the (mutated) solution; Bob verifies it for correctness



How To Convince Bob the Puzzle Has a Solution?

Step 5: Alice permutes the solution again to get a different set of numbers (still mapped from the original solution)

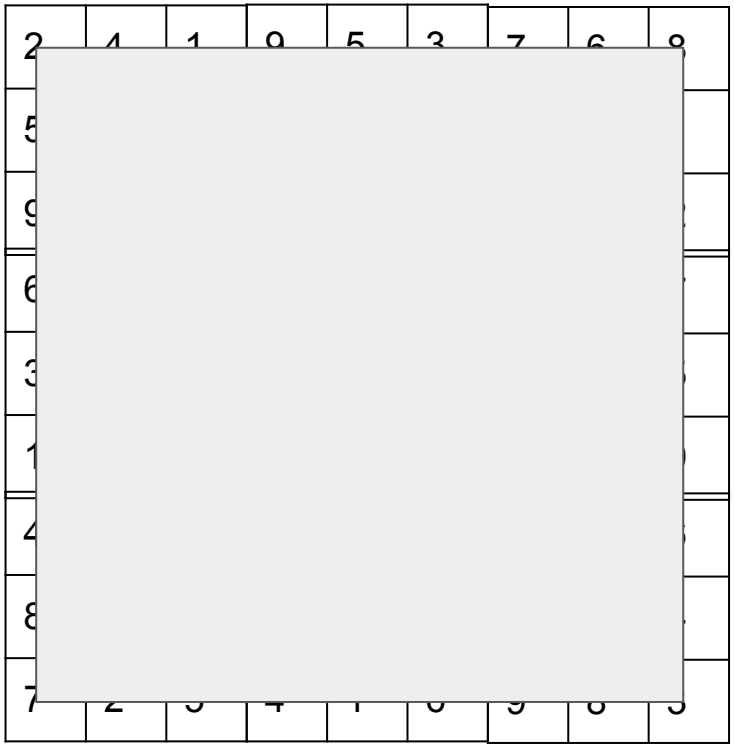
9	3	2	1	4	8	5	6	7
4	6	8	7	5	9	3	1	2
1	7	5	6	3	2	4	8	9
6	1	9	2	7	4	8	3	5
8	5	3	9	6	1	7	2	4
2	4	7	5	8	3	6	9	1
3	8	1	4	9	7	2	5	6
7	2	6	8	1	5	9	4	3
5	9	4	3	2	6	1	7	8



2	4	1	9	5	3	7	6	8
5	6	3	8	7	2	4	9	1
9	8	7	6	4	1	5	3	2
6	9	2	1	8	5	3	4	7
3	7	4	2	6	9	8	1	5
1	5	8	7	3	4	6	2	9
4	3	9	5	2	8	1	7	6
8	1	6	3	9	7	2	5	4
7	2	5	4	1	6	9	8	3

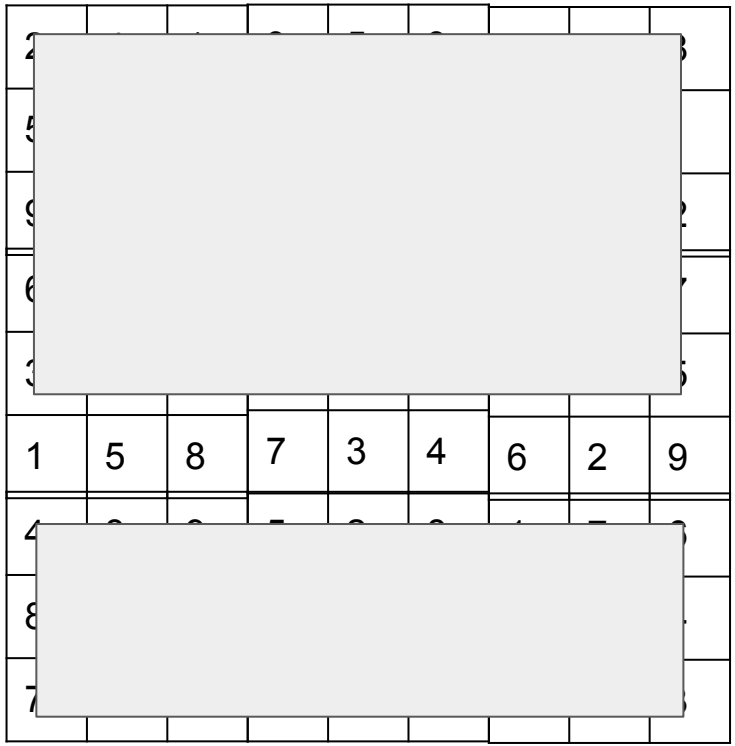
How To Convince Bob the Puzzle Has a Solution?

Step 6: the new masked solution is presented to Bob



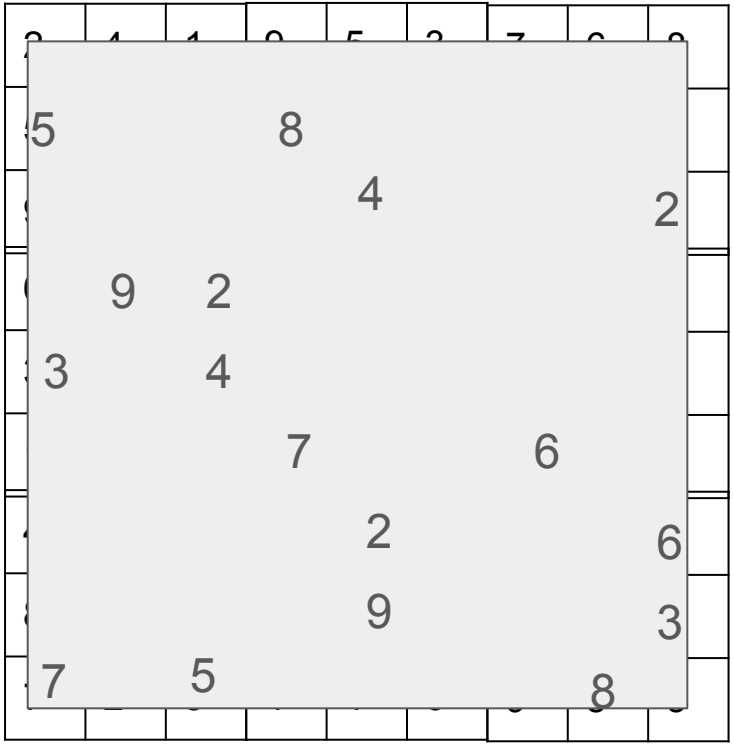
How To Convince Bob the Puzzle Has a Solution?

Step 7: Bob randomly picks another unit and asks Alice to reveal the (mutated) solution; Bob verifies it for correctness



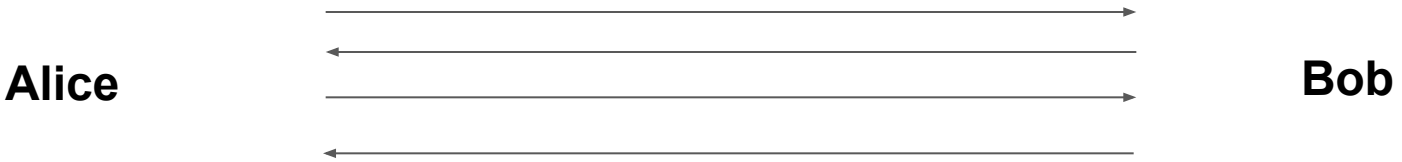
How To Convince Bob the Presented Solution Is For the Right Puzzle?

Step 8: for any round, Bob could also ask for the original positions of the puzzle to be disclosed. Seeing the scrambled numbers, Bob can be convinced that it's properly mapped from the original public positions



How To Convince Bob the Puzzle Has a Solution?

Repeat the procedure **(mutate -> mask -> pick unit) n times**, if Alice gets it right every time, Bob knows that the chance she achieved this by cheating without a valid solution, is at most $\mathbf{(27/28)^n}$. For **n=200,** the chance is **~ 0.05%**



The above protocol for Zero Knowledge Proof is a **probabilistic** proof

Simplest form of ZKP

NOT based on SNARK

Alice To Prove She Knows a Secret Key

A secret key in the world of elliptic curve cryptography is a large random number

Let the secret key be **a**.

The corresponding public key is **$PK = g^a \bmod P$**

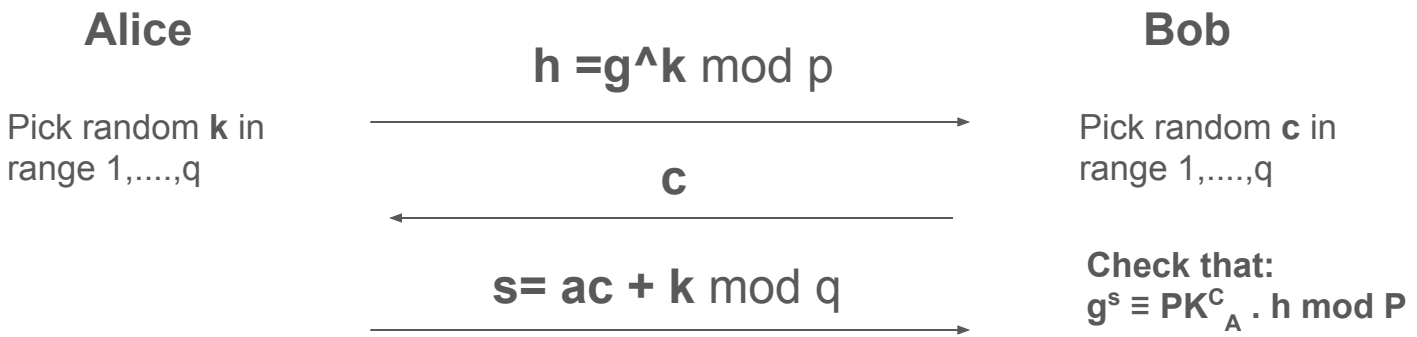
We know from math that:

- **$g^{(ac)} = (g^a)^c = (PK)^c$**
- **$g^{(m+k)} = g^m . g^k$**

We also know that given **$g^a \bmod P$**, it's computationally infeasible to reverse engineer **a** if **a** is sufficiently large.

Alice To Prove She Knows a Secret Key

Using the same idea as how Alice proves to Bob she knows the Sudoku solution, she asks Bob to pick a random large number c, so she can prove to Bob she knows the secret a, without telling Bob what a is, by presenting s=ac+k.



Recall that:

$g^s = g^{(ac+k)} = (g^a)^c \cdot g^k$

And that the public key $PK_A = g^a$ is public knowledge

The Construction of SNARK

Time To Get Technical

We now look at one of the most popular zero knowledge proof schemes: **SNARK (succinct non-interactive argument of knowledge).**

Given a function **f(x)**, and public output **y**, using **zkSNARK**, one can generate a proof to demonstrate the knowledge of a solution **s**, without revealing the value of **S**.

Given: **f(x) = y**

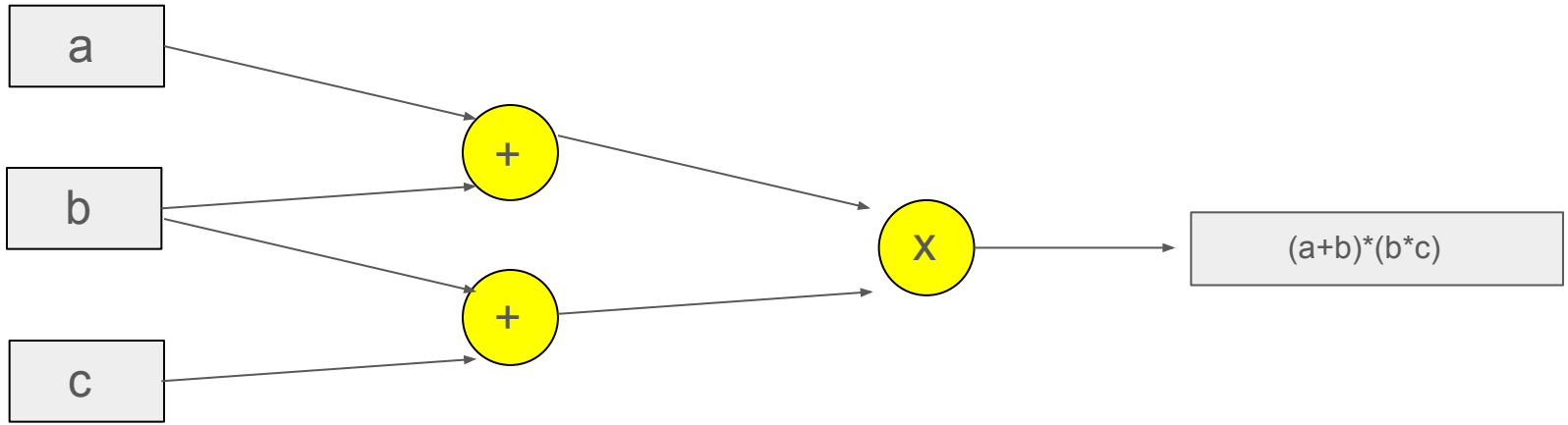
Produce **s**, such as that **f(s) = y**

SNARK consumes the "code" of the function **f(x)** and public input **y** as input, and produces the zk proof as the output.

How To Capture Performed Computations

We use "circuits" to model computations. Think of a circuit as a network of "gates" with two input wires and an output wire:

For (a+b) * (b*c):



R1CS Representation of the circuit

Using a **Rank 1 Constraint System (RCS)**, the computations can be captured in a collection of vectors.

R1CS is a vector based system that represents the sequence of gates in the circuit.

The circuit that calculates **y = x**3 + 2x + 5** can be broken down into gates:

1) T1 =x *x

2) z =T1 *x

3) T2 = z + 2x

4) y = T2 + 5

R1CS Representation of the circuit

The next step is building a "witness expression", that captures all the variables involved in the computation gates. For the gates involved in the above calculation,

the witness expression is: **w = (1, x, y, T1, z, T2)**

We then use the expression to represent the state of each of the input and output wires of each gate:

A, B are the input wires

C is the output wire

1) T1 =x *x

2) z =T1 *x

3) T2 = (z + 2x)* 1

4) y = (T2 + 5) * 1

	(1, X, Y, T1, z, T2)	(1, X, Y, T1, z, T2)	(1, X, Y, T1, z, 12)
	A	B	C
1)	[0, 1, 0, 0, 0, 0]	[0, 1, 0, 0, 0, 0]	[0, 0, 0, 1, 0, 0]
2)	[0, 0, 0, 1, 0, 0]	[0, 1, 0, 0, 0, 0]	[0, 0, 0, 0, 1, 0]
3)	[0, 2, 0, 0, 1, 0]	[1, 0, 0, 0, 0, 0]	[0, 0, 0, 0, 0, 1]
4)	[5, 0, 0, 0, 0, 1]	[1, 0, 0, 0, 0, 0]	[0, 0, 1, 0, 0, 0]

Verifying R1CS

An R1CS matrix can be verified using the "°" **operator** with a "**witness**":

w°A * w°B = w°C

Using the example: **P(x) = x**3 + x + 5,** P(3) = 3**3 + 3 + 5 = 35,

Expressed in witness format: [1, 3, 35, 9, 27, 30]

(1, 3, 35, 9, 27, 30)		(1, 3, 35, 9, 27, 30)		(1, 3, 35, 9, 27, 30)	
A		B		C	
[0, 1, 0, 0, 0, 0]		[0, 1, 0, 0, 0, 0]		[0, 0, 0, 1, 0, 0]	3 * 3 = 9
[0, 0, 0, 1, 0, 0]		[0, 1, 0, 0, 0, 0]		[0, 0, 0, 0, 1, 0]	9 * 3 = 27
[0, 2, 0, 0, 1, 0]	*	[1, 0, 0, 0, 0, 0]	=	[0, 0, 0, 0, 0, 1]	(3 + 27) * 1 = 30
[5, 0, 0, 0, 0, 1]		[1, 0, 0, 0, 0, 0]		[0, 0, 1, 0, 0, 0]	(5 + 30) * 1 = 35

"Compress" The Verifier's Task

Now that all the computations performed by the prover have been captured, the verifier just need to check each of the steps in the computation.

But is it possible to make the verification faster? It'd be ideal to have a protocol that's easier to verify than to compute.

We can use another transformation with **Quadratic Arithmetic Programs** to turn the result of the R1CS vectors into polynomial expressions.

$$P(x) = c_0 + c_1x^1 + c_2x^2 + \ldots + c_dx^d$$

Why Polynomials?

Two interesting properties about polynomials are useful:

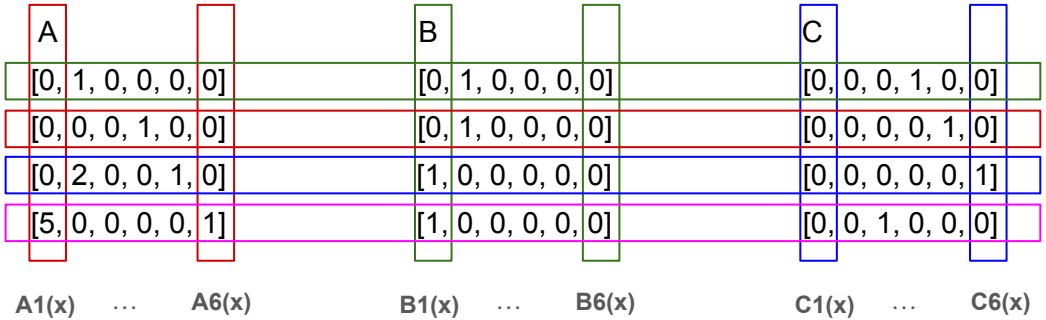
- One expression can embed infinite amount of information

- The coefficients of a polynomial can represent arbitrary information

- Comparing the knowledge of the set of coefficients of a polynomial is easy

- Just plug in a few **x** values, if Alice is able to return the expected result, then she must know the set of coefficients

R1CS -> QAP: Representing the 4 “steps” (constraints) of the calculation



4 sets of vector groups, 3 vectors per group.

Converted to 3 groups of polynomials of degree-3, 6 polynomials per group

$A1(x) = 0.833x^3 - 5x^2 + 9.166x - 5$
$A2(x) = -0.666x^3 + 5x^2 - 11.333x + 8$
. . .
$B1(x) = -0.333x^3 + 2.5x^7 - 5.166x + 3.0$
$B2(x) = -0.333x^3 + 2.5x^2 - 5.166x + 3.0$
.. .
$C1(x) = 0x^3 + 0x^2 + 0x + 0$
$C2(x) = 0x^3 + 0x^2 + 0x + 0$

Constraints for finding the polynomials

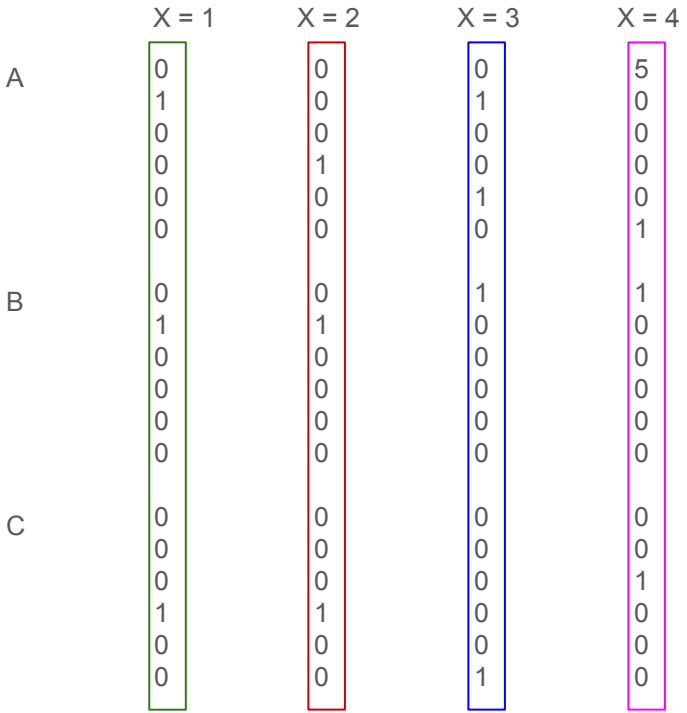
	x = 1	x = 2	x = 3	x = 4
$A1(x) = 0.833x^3 - 5x^2 + 9.166x - 5$	0	0	0	5
$A2(x) = -0.666x^3 + 5x^2 - 11.333x + 8$	1	0	1	0
$A3(x) = 0x^3 + 0x^2 + 0x + 0$	0	0	0	0
$A4(x) = 0.5x^3 - 4x^2 + 9.5x - 6$	0	1	0	0
$A5(x) = -0.5x^3 + 3.5x^2 - 7x + 4$	0	0	1	0
$A6(x) = 0.166x^3 - x^2 + 1.833x - 1$	0	0	0	1
$B1(x) = -0.333x^3 + 2.5x^2 - 5.166x + 3$	0	0	1	1
$B2(x) = 0.333x^3 - 2.5x^2 + 5.166x - 2$	1	1	0	0
$B3(x) = 0x^3 + 0x^2 + 0x + 0$	0	0	0	0
$B4(x) = 0x^3 + 0x^2 + 0x + 0$	0	0	0	0
$B5(x) = 0x^3 + 0x^2 + 0x + 0$	0	0	0	0
$B6(x) = 0x^3 + 0x^2 + 0x + 0$	0	0	0	0
$C1(x) = 0x^3 + 0x^2 + 0x + 0$	0	0	0	0
$C2(x) = 0x^3 + 0x^2 + 0x + 0$	0	0	0	0
$C3(x) = 0.166x^3 - x^2 + 1.833x - 1$	0	0	0	1
$C4(x) = -0.166x^3 + 1.5x^2 - 4.333x + 4$	1	1	0	0
$C5(x) = 0.5x^3 - 4x^2 + 9.5x - 6$	0	0	0	0
$C6(x) = -0.5x^3 + 3.5x^2 - 7x + 4$	0	0	1	0

R1CS -> QAP

R1CS:

A	B	C
[0, 1, 0, 0, 0, 0]	[0, 1, 0, 0, 0, 0]	[0, 0, 0, 1, 0, 0]
[0, 0, 0, 1, 0, 0]	[0, 1, 0, 0, 0, 0]	[0, 0, 0, 0, 1, 0]
[0, 2, 0, 0, 1, 0]	[1, 0, 0, 0, 0, 0]	[0, 0, 0, 0, 0, 1]
[5, 0, 0, 0, 0, 1]	[1, 0, 0, 0, 0, 0]	[0, 0, 1, 0, 0, 0]

QAP:



Verifying the computation with the polynomials

Just as how we used the witness [1, 3, 35, 9, 27, 30] to verify the R1CS matrix, we can use the witness to verify the polynomials.

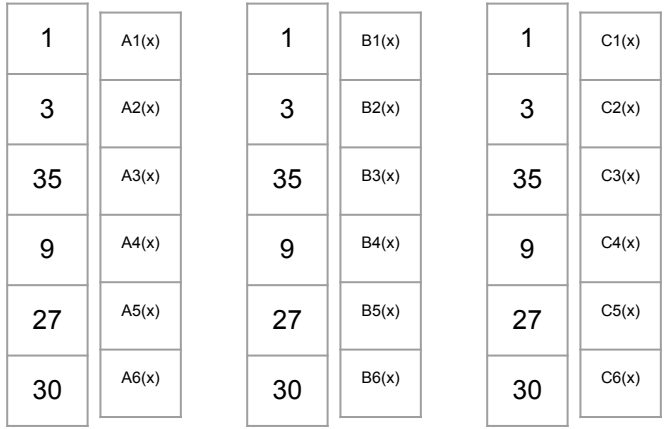
The polynomials resulted from the QAP transformation also satisfy the "o" operator with the witness:

$$A(x) \circ \mathbf{w} * B(x) \circ \mathbf{w} - C(x) \circ \mathbf{w} = H(x) * Z(x)$$

Z(x) is the "zero polynomial", where it's zero at all x values, for x in [1..n] where n is the number of gates:

(x-1) * (x-2) * (x-3) ...

Checking our work



1	$A1(x) = 0.833x^3 - 5x^2 + 9.166x - 5$
3	$A2(x) = -0.666x^3 + 5x^2 - 11.333x + 8$
35	$A3(x) = 0x^3 + 0x^2 + 0x + 0$
9	$A4(x) = 0.5x^3 - 4x^2 + 9.5x - 6$
27	$A5(x) = -0.5x^3 + 3.5x^2 - 7x + 4$
30	$A6(x) = 0.166x^3 - x^2 + 1.833x - 1$

$A(x) = -5.166x^3 + 38.5x^2 - 73.333x + 43$

$A(x) \quad * \quad B(x) \quad - \quad C(x) \quad = H * Z(x)$

1	$B1(x) = -0.333x^3 + 2.5x^2 - 5.166x + 3$
3	$B2(x) = 0.333x^3 - 2.5x^2 + 5.166x - 2$
35	$B3(x) = 0x^3 + 0x^2 + 0x + 0$
9	$B4(x) = 0x^3 + 0x^2 + 0x + 0$
27	$B5(x) = 0x^3 + 0x^2 + 0x + 0$
30	$B6(x) = 0x^3 + 0x^2 + 0x + 0$

$B(x) = 0.666x^3 - 5x^2 + 10.333x - 3$

1	$C1(x) = 0x^3 + 0x^2 + 0x + 0$
3	$C2(x) = 0x^3 + 0x^2 + 0x + 0$
35	$C3(x) = 0.166x^3 - x^2 + 1.833x - 1$
9	$C4(x) = -0.166x^3 + 1.5x^2 - 4.333x + 4$
27	$C5(x) = 0.5x^3 - 4x^2 + 9.5x - 6$
30	$C6(x) = -0.5x^3 + 3.5x^2 - 7x + 4$

$C(x) = 2.833x^3 - 24.5x^2 + 71.666x - 41$

Checking our work

$A(x) = -5.166x^3 + 38.5x^2 - 73.333x + 43$

$B(x) = 0.666x^3 - 5x^2 + 10.333x - 3$

$C(x) = 2.833x^3 - 24.5x^2 + 71.666x - 41$

$$(-5.166x^3 + 38.5x^2 - 73.333x + 43)$$

$$* \quad (0.666x^3 + 5x^2 + 10.333x - 3)$$

$$(-3.444x^6 + 51.5x^5 - 294.777x^4 + 808.666x^3 - 1088.277x^2 + 664.333x - 129)$$

$$- (2.833x^3 - 24.500x^2 + 71.666x - 41)$$

$$-3.444x^6 + 51.5x^5 - 294.77781x^4 + 805.83383x^3 - 1063.7778x^2 + 592.666x - 88$$

$A(x) * B(x) - C(x) = -3.444x^6 + 51.5x^5 - 294.777x^4 + 805.833x^3 - 1063.777x^2 + 592.666x - 88$

Checking our work

$A(x) = (x-1)(x-2)(x-3)(x-4) = x^4 - 10x^3 + 35x^2 - 50x + 24$

$$\frac{-3.444x^6 + 51.5x^5 - 294.77781x^4 + 805.83383x^3 - 1063.7778x^2 + 592.666x - 88}{x^4 - 10x^3 + 35x^2 - 50x + 24} = -3.444x^2 + 17.055x - 3.666$$

(no remainder!)

$A(x) * B(x) - C(x)$

$= -3.444x^6 + 51.5x^5 - 294.777X^4 + 805.833x^3 - 1063.777x^2 + 592.666x - 88$

$= (-3.444x^2 + 17.055x - 3.666) * (x^4 - 10x^3 + 35x^2 - 50x + 24)$

$= H(x) * Z(x)$

When Proving and Verifying…

Common Knowledge shared b/w Alice (prover) and Bob (verifier):

- **A1(x), A2(x) ... An(x)**
- **B1(x), B2(x) ... Bn(x)**
- **C1(x), C2(x) ... Cn(x)**
- **Z(x)**

Prover:

- Performs the calculations with the circuit and records the witness v
- Uses **w** and calculates:
    - A(x) = w ° {A1(x), A2(x) ... An(x)} = 1 * A1(x) + w2 * A2(x) + w3 * A3(x) + ...
    - B(x) = w ° {B1(x), B2(x) ... Bn(x)} = 1 * B1(x) + w2 * B2(x) + w3 * B3(x) + ...
    - C(x) = w° {C1(x), C2(x) ... Cn(x)} = 1 * C1(x) + w2 * C2(x) + w3 * C3(x) + ...
- Then calculates **H(x) = (A(x) * B(x) - C(x)) / Z(x)**, and evaluates **A(x), B(x), C(x), H(x)** at point s, randomly chosen by the verifier. Shares A(s), B(s), C(s), H(s) with the verifier

Verifier:

- Checks that **A(s) * B(s) - C(s) = H(s) * Z(s) holds**. This proves all the constraints of the circuits have been met, meaning all the calculations are done correctly according to the circuit definition

Almost There

With the transformations so far, the problem becomes this:

- Alice needs to prove to Bob she knows a set of values, called the "witness", that is the result of correct computation according to the circuit.

- Using her "witness" values and the QAP set of polynomials (public knowledge derived from the circuit), Alice calculates polynomials **A(x), B(x), C(x) and H(x)**

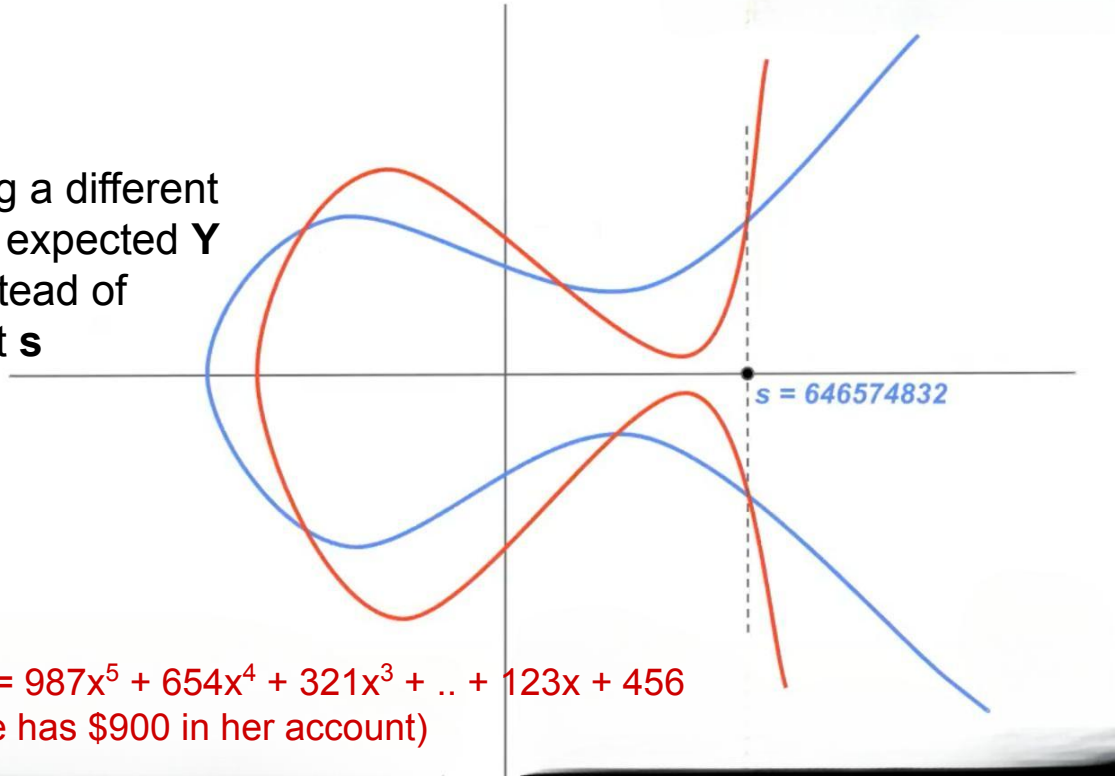
- By providing **A(s), B(s), C(s)** and **H(s)**, Alice can prove that she performed the calculations correctly (without cheating)

- **But**: Alice must not know the value **s**, Bob must not know the polynomials

Bob Must Hide The Evaluation Points from Alice

$P(x) = 123x^5 + 456x^4 + 789x^3 + \ldots + 321x + 654$
(Alice has $1000 in her account)

Otherwise Alice can cheat by finding a different polynomial that produces the same expected **Y** value with her invalid input (900 instead of 1000), ONLY at the evaluation point **s**



$P(x) = 987x^5 + 654x^4 + 321x^3 + .. + 123x + 456$
(Alice has $900 in her account)

Techniques for Hiding Shared Evaluation Parameters

- Homomorphic Hiding
- Blind Evaluation of Polynomials

Given the challenge: **$P(x) = 123x^3 + 456x^2 + 789x + 5$**
(Alice has $1000 in her account)
Bob picks random evaluation point **S**
Calculates the hidings: $\{g^{s3}, g^{s2}, g^s, g\}$, s3 = s3, $s2 = s^2$, ...

Having the proof: $P(x) = 123x^3 + 456x^2 + 789x + 5$
(Alice has $1000 in her account)

Using the hidings for the evaluation point shared by Bob: $\{g^{s3}, g^{s2}, g^s, g\}$

Calculates: $g^P(s) = (g^{s3})^{123} * (g^{s2})^{456} * (g^s)^{789} * g^5$

Bob can check Alice's knowledge of the polynomial without giving out the point **S**

Convert to Non-interactive Proof

- using a Common Reference String (CRS) obtained from a Trusted Setup

- Contains the proving key and the verifying key

    - **Proving key:** hidings of **A1(s), A2(s)**..., and the same for **B** and **C**, s is the secret evaluation point

    - **Verifying key:** hidings related to the public inputs

Usage Scenarios of ZKP

Usage of ZKPs

- Digital Signatures (eg. ECDSA) is essentially a zero knowledge proof of the knowledge of the private key

Usage of ZKPs

- Hide transaction payloads in a blockchain
  - Alice sends Bob 10 fungible tokens, by executing the transaction privately and sends a ZKP of the tx
  - All validators in the blockchain network can see the zkp, and verify that the transaction has been executed honestly, without knowing the details of the transaction
  - zCash, EY Nightfall confidential token

Usage of ZKPs

- Layer-2 Scaling Solutions
  - ZKP is great at compressing a large amount of computation into a small proof that can be verified fast
  - zkSync, Loopring, StarkNet, Polygon zkEVM, etc.

Usage of ZKPs

- Self-Sovereign Identities
  - Presenting a claim about "are you over 21 years old to be admitted into the bar?" without disclosing the birth date, but by using a ZKP

Where to Learn More

Best learning websites with curated content:

- https://learn.Oxparc.org/

Step-by-Step Explanation of SNARK (without skipping the math):

- https://electriccoin.co/blog/snark-explain/

Another Step-by-Step Explanation

- https://www.zeroknowledgeblog.com/

101 Level hands-on tutorials:

- https://github.com/privacy-scaling-explorations/zkp-app-boilerplate